TECHNICAL REFERENCE · MULTI-AGENT PROCUREMENT INTELLIGENCE

Inside InsightIQ

A complete walkthrough of how four cooperating agents turn a raw inventory ledger into an auditable, one-click purchase order — every formula, every rule, and every decision boundary, documented against the running system.

AGENTS	SOURCE DATASETS	ROWS INGESTED	SUPPLIER POOL	FORWARD-CHAINING RULES	URGENT SKUS
4 + 1	6	94,962	10	4	53

01

What InsightIQ does

InsightIQ is a procurement decision support system. It watches an inventory ledger, notices when a product is running out, works out who should supply the replacement stock, checks that supplier against a set of business rules, and produces a costed purchase order with a delivery date — then explains, step by step, why it made every one of those choices.

The work is split across **four specialist agents** arranged in a pipeline, coordinated by a fifth. Each agent has one job and hands a structured result to the next. Nothing is a black box: the Knowledge Agent emits a full reasoning trace that the interface replays as an animation, so a human reviewer can follow the logic rule by rule before approving anything.

> THE CORE IDEA
>
> Ranking and risk are deliberately kept apart. The Supplier Intelligence Agent answers *"who scores best?"* using a weighted heuristic. The Knowledge Agent independently answers *"does this supplier break any rules?"* A high score can never override a risk rule — that separation is what makes the recommendation defensible.

02

System architecture

The **Coordinator Agent** is the system controller. It runs the four agents in sequence, times each stage, and bundles every intermediate result into a single response — so the

pipeline view, the explainability inspector and the action hub all read from one payload rather than making separate calls.

STAGE 1	STAGE 2	STAGE 3
Perception Reads the inventory ledger. Detects shortfall, computes the deficit.	**Supplier Intelligence** Filters eligible suppliers, scores and ranks them by heuristic search.	**Knowledge** Forward-chains IF–THEN rules to assign each supplier a status.

STAGE 4

Planning

Builds the order: quantity, arrival date, cost, backup supplier.

Each stage records its own timing and a one-line summary. A representative run of `SKU-0717` on the live system:

STAGE	DURATION	REPORTED OUTCOME
Perception	13.7 ms	Scanned inventory ledger — 25 urgent SKUs detected
Supplier Intelligence	8.0 ms	Ranked 10/10 eligible suppliers via heuristic search
Knowledge	0.1 ms	Forward-chained 27 inference steps; recommended SUP_1
Planning	2.0 ms	Plan ready — 2,640 units, arriving 2026-08-28

The whole pipeline completes in roughly **24 milliseconds**. All reasoning is deterministic and rule-based — there is no machine learning model and no external AI call anywhere in the decision path, which is precisely why every output can be traced back to a formula or a rule.

03

The data foundation

Six raw CSV files are cleaned and materialised into a local SQLite database on first run. Derived columns are computed during ingestion so the agents can query fast.

TABLE	ROWS	PURPOSE
products	1,000	Stock levels, reorder points, prices — the Perception Agent's sensor
orders	1,000	Historical fulfilment and lead-time signals

TABLE	ROWS	PURPOSE
customers	1,000	Outbound demand signals by country
calendar	935	Working days and seasonality for delivery projection
procurement_kpi	777	Vendor quality history: defect rate, compliance, cost variance
supply_chain	91,250	Live logistics telemetry: cost, lead time, stock on hand
supplier_profiles	10	*Derived* — the unified supplier pool the agents actually score
purchase_orders	—	Approved orders written back by the Action Hub

Bridging two disconnected supplier worlds

The two supplier datasets do not share a join key, and each is missing what the other has. The logistics feed tracks ten numeric suppliers (`SUP_1` … `SUP_10`) with live cost, stock and lead-time data but *no quality signal*. The procurement KPI file tracks five named vendors with rich quality history but *no live pricing or stock*.

Ingestion reconciles them by deterministically round-robin assigning each live supplier a named vendor as its quality benchmark, producing one pool that carries both. Three fields are synthesised in the process:

```
— Quality rating, from the KPI vendor's history (clipped to 1–5)
Rating = 5 × (0.4·FulfillmentRate + 0.3·ComplianceRate + 0.3·(1 − Def

— Reliability, from how *consistent* the supplier's lead time is
  (low variance = dependable). Min–Max inverted, then clipped to 0.5–
Reliability = 1 − MinMax(LeadTimeStdDev)

— Delivery estimate, blending live telemetry with historical KPI reco
DeliveryTime = 0.7 × LiveAvgLeadTime + 0.3 × HistoricalDeliveryDays
```

WORTH KNOWING

Because reliability is clipped to a 0.5–1.0 floor, four of the ten suppliers sit at exactly `0.500` . They are indistinguishable on that axis, and only one supplier (`SUP_4` , at 1.000) clears the Knowledge Agent's 0.90 reliability bar. That is a property of how the source data is generated, not a scoring fault.

04

Perception Agent

The Perception Agent observes the environment and emits structured facts. It answers one question: **which products need replenishing, and by how much?** It makes no decisions — it only reports state.

FORMULAS

```
TargetStock = ReorderLevel × 3        – cover lead-time demand plus s
Deficit     = max(0, TargetStock – UnitsInStock)
Urgent      = UnitsInStock < ReorderLevel
```

The multiplier of **3** encodes the policy that healthy stock is three times the reorder point. Note the two thresholds do different jobs: `Urgent` decides *whether* to act, while `Deficit` decides *how much* to buy — and it is measured against the target, not the reorder point, so an order restores a comfortable buffer rather than scraping the minimum.

WORKED EXAMPLE – SKU-0717 “HEADPHONES”

```
Inventory     = 144 units
ReorderPoint  = 928 units

TargetStock   = 928 × 3        = 2,784
Deficit       = 2,784 – 144    = 2,640 units
Urgent        = 144 < 928       = TRUE → flagged for procurement
```

Alongside the focus SKU, the agent scans the full ledger (currently **53 urgent SKUs**) and aggregates demand signals — top buyers, demand by country, and a twelve-month order trend — which feed the dashboard.

05

Supplier Intelligence Agent

This agent performs the **heuristic search**. It takes the deficit from Perception, narrows the supplier pool to those that can actually fill it, scores each one on four weighted attributes, and returns a ranked candidate list. This is the mathematical core of the system, so it is worth taking slowly.

Step 1 — Eligibility filtering

A supplier that cannot cover the order is not scored at all. Filtering happens *before* any normalisation, which matters for the step that follows.

```
IF stock_availability ≥ Deficit  →  eligible
ELSE                             →  excluded
```

If *no* supplier can single-handedly cover the deficit, the agent falls back to ranking the entire pool rather than failing, so a partial or split-order strategy remains possible downstream.

Step 2 — Min–Max normalisation over the eligible pool only

The four attributes are measured in incompatible units — rupees, days, stars, a ratio. They are made comparable with Min–Max normalisation, which rescales any value to a 0–1 range:

```
N(x) = (x − Min(x)) / (Max(x) − Min(x))
```

Min and Max are taken from the eligible pool, not the full pool. A supplier that was filtered out cannot stretch the scale — scores always express "how good is this supplier compared with the ones actually in contention."

Step 3 — Direction correction

Raw normalisation always maps the minimum to 0 and the maximum to 1, but the minimum isn't always the *best*. Cheap is good; slow is bad. Before weighting, every component is flipped where necessary so that all four point the same way:

COMPONENT	DIRECTION	FORMULA	SCORE OF 0	SCORE OF 1
Rating	Higher is better	`Rating / 5`	Lowest rated	Highest rated
Cost	Lower is better	`1 − N(Cost)`	Most expensive	Cheapest
Reliability	Higher is better	`N(Reliability)`	Least reliable	Most reliable
Delivery	Lower is better	`1 − N(DeliveryTime)`	Slowest	Fastest

Rating is the exception that is *not* Min–Max normalised. It already occupies a fixed, externally meaningful 0–5 scale, so it is simply divided by 5. Normalising it would make a 3-star supplier score 0 purely because it happened to be the worst in a strong field.

Step 4 — The weighted Raw Score

The four direction-corrected components are combined with fixed business weights. Cost and Reliability together account for exactly half the score:

```
RawScore = 0.30 × RatingScore
         + 0.25 × CostScore
         + 0.25 × ReliabilityScore
         + 0.20 × DeliveryScore

Fully expanded:

RawScore = 0.30 × (Rating / 5)
         + 0.25 × (1 − (Cost − MinCost) / (MaxCost − MinCost))
         + 0.25 × ((Rel − MinRel) / (MaxRel − MinRel))
         + 0.20 × (1 − (Dlv − MinDlv) / (MaxDlv − MinDlv))

Weights total 1.00, so RawScore is bounded 0 … 1
```

PRIORITY	WEIGHT	REASONING
Rating	30%	Overall vendor quality carries the most weight
Cost	25%	Together exactly **50%** — commercial terms and dependability are the deciding pair
Reliability	25%	
Delivery	20%	Speed matters, but least of the four

Step 5 — The Final Score (0–100)

Raw Scores are rescaled once more, to a presentation-friendly 0–100 band:

```
FinalScore = ((RawScore − MinRawScore) / (MaxRawScore − MinRawScore))
```

READ THIS CORRECTLY

Final Score is a **relative ranking score**, nothing more. Because it is rescaled across the pool, the best supplier always scores exactly 100 and the worst exactly 0 — regardless of how good either actually is. It is not a safety score, not a risk probability, and not a percentage chance of success. Risk is the Knowledge Agent's job.

Edge cases

When a pool cannot be discriminated on some axis — every supplier charges the same, or there is only one eligible supplier — the denominator becomes zero. Each case has a defined answer rather than a crash:

CONDITION	RESULT	WHY
`MaxCost == MinCost`	`CostScore = 1.0` for all	Cost can't differentiate, so nobody is penalised for it
`MaxRel == MinRel`	`ReliabilityScore = 1.0` for all	Same reasoning
`MaxDlv == MinDlv`	`DeliveryScore = 1.0` for all	Same reasoning
`MaxRaw == MinRaw`	`FinalScore = 100` for all	No supplier can be ranked above another
One eligible supplier	`FinalScore = 100`, rank 1	Every axis is degenerate at once
Missing / null / infinite input	Supplier excluded, reported separately	Values are never invented to fill a gap

The live supplier pool, fully scored

All ten suppliers with their component scores and resulting rank. The top five are passed to the Knowledge Agent.

#	SUPPLIER	COST	RATING	DELIVERY	RELIABILITY	R ·30	C ·25	RL ·25	D ·20	RAW	FINAL
1	**SUP_5**	₹10.96	4.16	7.9d	0.872	0.832	0.974	0.744	1.000	0.8791	100.0
2	**SUP_1**	₹11.85	4.39	8.9d	0.752	0.878	0.710	0.504	0.286	0.6240	60.7
3	**SUP_9**	₹10.88	4.37	9.0d	0.597	0.874	1.000	0.194	0.214	0.6036	57.5
4	**SUP_4**	₹13.26	4.37	9.3d	1.000	0.874	0.286	1.000	0.000	0.5837	54.5
5	**SUP_6**	₹12.88	4.39	8.7d	0.758	0.878	0.400	0.516	0.429	0.5780	53.6
6	**SUP_10**	₹12.53	4.16	8.2d	0.572	0.832	0.506	0.144	0.786	0.5693	52.3
7	**SUP_7**	₹11.64	3.90	8.7d	0.500	0.780	0.770	0.000	0.429	0.5123	43.5

#	SUPPLIER	COST	RATING	DELIVERY	RELIABILITY	R ·30	C ·25	RL ·25	D ·20	RAW	FINAL
8	**SUP_2**	₹11.39	3.90	9.3d	0.500	0.780	0.845	0.000	0.000	0.4453	33.1
9	**SUP_3**	₹11.97	3.60	9.0d	0.500	0.720	0.672	0.000	0.214	0.4269	30.3
10	**SUP_8**	₹14.22	3.60	9.2d	0.500	0.720	0.000	0.000	0.071	0.2303	0.0

VERIFYING THE WINNER BY HAND

```
SUP_5 — Rating 4.16, Cost ₹10.9643, Delivery 7.9d, Reliability 0.872
Pool ranges: Cost ₹10.8769–₹14.2164 · Delivery 7.9–9.3d · Reliability

RatingScore      = 4.16 / 5                                  = 0.832
CostScore        = 1 - (10.9643-10.8769)/(14.2164-10.8769)   = 0.974
ReliabilityScore = (0.872 - 0.500) / (1.000 - 0.500)         = 0.744
DeliveryScore    = 1 - (7.9 - 7.9) / (9.3 - 7.9)             = 1.000

RawScore = 0.30(0.832) + 0.25(0.974) + 0.25(0.744) + 0.20(1.000)
         = 0.2496 + 0.2435 + 0.1860 + 0.2000
         = 0.8791   → highest in pool → FinalScore 100.0
```

SUP_5 wins on breadth: the fastest delivery in the pool, near-cheapest cost, and second-best reliability. Note that `SUP_9` is *cheaper* (cost score 1.000 versus 0.974) yet ranks third — its weak reliability (0.194) and slow delivery (0.214) cost it more than the cost advantage gains.

06

Knowledge Agent

The Knowledge Agent applies **forward chaining** — the classic data-driven inference strategy from expert systems. It starts from known facts, finds every rule whose conditions those facts satisfy, fires them to derive new facts, then sweeps the rule set again. It repeats until a pass produces nothing new: a **fixed point**.

```
working memory ← facts from Perception + Supplier Intelligence

REPEAT
    FOR each rule not yet fired:
        IF its conditions hold against working memory:
            fire it → assert the new fact
UNTIL no rule fires in a full pass        (fixed point reached)
```

Every firing — matched or not — is appended to a structured reasoning trace, which is what makes the decision auditable and what the XAI inspector animates.

The rule set

RULE 1 — INVENTORY STATE

```
IF   Inventory < ReorderPoint
THEN State = "Urgent Procurement"
ELSE State = "Monitoring"
```

RULE 2 — PREFERRED SUPPLIER

```
IF   Rating ≥ 4.5  AND  DeliveryTime ≤ 3 days
THEN Supplier_Status = "Preferred"
```

Both conditions must hold. This is a demanding bar, and no supplier in the current pool meets it — the best rating is 4.39 and the fastest delivery is 7.9 days.

RULE 3 — RISK ASSESSMENT

```
IF   DefectRate > 0.05        (5%)
THEN Supplier_Status = "High Risk"
```

A strict inequality, which makes the boundary sharp: a defect rate of exactly 0.05 does *not* trigger the rule, while 0.0502 does. This precision is preserved end to end — the agent receives unrounded values specifically so display formatting can never flip a verdict.

RULE 4 — RELIABILITY WITH FALLBACK

```
IF   no prior status  AND  Reliability ≥ 0.90
THEN Supplier_Status = "Reliable"
ELSE Supplier_Status = "Standard"
```

Rule 4 only evaluates if Rules 2 and 3 both left the supplier unlabelled — the `no prior status` guard. It is the safety net that guarantees every supplier ends up with a status, defaulting to *Standard*.

Status precedence

A supplier can accumulate more than one label in a sweep. Precedence resolves it, and **High Risk always wins**:

```
High Risk  ▸  Preferred  ▸  Reliable  ▸  Standard
```

A complete trace — SUP_5

The top-ranked supplier, evaluated rule by rule. Notice the chain stops early:

```
FACT        SUP_5 — Rating 4.16 · Delivery 7.9d · DefectRate
            0.0502 · Reliability 0.872
  ↓
RULE 2      4.16 ≥ 4.5 → FALSE  AND  7.9 ≤ 3 → FALSE  ⇒  not
            matched
  ↓
RULE 3      0.0502 > 0.05 → TRUE ⇒ MATCHED — Supplier_Status =
            "High Risk"
  ↓
RULE 4      Skipped — a prior status now exists, so the guard
            fails
  ↓
DECISION    SUP_5 Supplier_Status = High Risk
```

By contrast, `SUP_1` runs the full chain and lands on the fallback branch:

```
FACT        SUP_1 — Rating 4.39 · Delivery 8.9d · DefectRate 0.0236 · Reliability 0.752
  ↓
RULE 2      4.39 ≥ 4.5 → FALSE  AND  8.9 ≤ 3 → FALSE  ⇒  not matched
  ↓
RULE 3      0.0236 > 0.05 → FALSE ⇒ not matched — no risk flag
  ↓
RULE 4      No prior status → TRUE  AND  0.752 ≥ 0.90 → FALSE ⇒ ELSE branch
  ↓
DECISION    SUP_1 Supplier_Status = Standard
```

All five candidates

RANK	SUPPLIER	DEFECT	RELIABILITY	RULE 2	RULE 3	RULE 4	STATUS
1	**SUP_5**	0.0502	0.872	no	MATCH	skipped	High Risk
2	**SUP_1**	0.0236	0.752	no	no	ELSE	Standard
3	**SUP_9**	0.0307	0.597	no	no	ELSE	Standard
4	**SUP_4**	0.0307	1.000	no	no	THEN	Reliable
5	**SUP_6**	0.0236	0.758	no	no	ELSE	Standard

07

Selecting the recommended supplier

This is where ranking and rules meet. The rule is deliberately simple:

> Walk the heuristic ranking from rank 1 downward.
> Recommend the **first candidate whose status is not "High Risk"**.

The heuristic proposes; the rule engine vetoes. A supplier can top the ranking and still be rejected, and no score is high enough to overrule a risk flag.

```
RANK 1          SUP_5 · score 100.0 · High Risk → rejected, continue
  ↓
RANK 2          SUP_1 · score 60.7 · Standard → RECOMMENDED
```

The backup supplier is chosen the same way: the highest-ranked remaining candidate that is *also* not High Risk — which is why the backup here is `SUP_9` (rank 3) and not `SUP_5`. Designating a rejected supplier as the contingency would contradict the very rule that disqualified it.

> **A NOTE ON THIS DATASET**
>
> The eligibility filter never actually excludes anyone here: the largest deficit across all 53 urgent SKUs is **2,723 units**, while the smallest supplier stock is **6,670**. Every supplier clears every SKU with room to spare, so the ranking is identical for every product. The filtering mechanism is correct and demonstrably active at higher deficits — this dataset simply never exercises it.

08

Planning Agent

The final agent converts a decision into an executable plan: how much to order, when it will arrive, what it will cost, and who to fall back on.

ORDER QUANTITY

```
OrderQuantity = roundUpToNearest10( max(Deficit, 10) )
```

Rounded up to a practical order multiple, with a floor so a trivial deficit still produces a viable order.

ARRIVAL DATE

Lead time is counted in **working days** — weekends are skipped, not merely subtracted — then a seasonality buffer is added. That buffer is derived from the calendar table rather than hard-coded: seasons with a higher density of non-working days are treated as higher-friction shipping periods and earn 0–2 extra working days.

```
baseLeadDays   = round(supplier.delivery_time_days)
naiveArrival   = addWorkingDays(today, baseLeadDays)      Mon–Fri only
finalArrival   = addWorkingDays(naiveArrival, seasonBuffer) 0–2 days
```

COST AND DEMAND PROJECTION

```
TotalExpense = OrderQuantity × supplier.avg_cost
DailyDemand  = max(1, round(ReorderPoint / 14))    reorder point ≈ 2
```

The daily demand estimate drives a **replenishment curve** — a day-by-day projection of stock depleting at the estimated rate, stepping back up on the arrival date — which the Action Hub charts so a reviewer can see whether the order lands before stock-out.

The plan for SKU-0717

```
OrderQuantity = roundUp(max(2640, 10))        = 2,640 units
baseLeadDays  = round(8.9)                     = 9 working days
naiveArrival  = 2026-08-15 + 9 working days    = 2026-08-27
season        = Summer                         → +1 day buffer
finalArrival  = 2026-08-28

DailyDemand   = max(1, round(928 / 14))        = 66 units/day
TotalExpense  = 2,640 × ₹11.846                = ₹31,273.44

Primary supplier : SUP_1   (rank 2 — rank 1 was High Risk)
Backup supplier  : SUP_9   (rank 3 — highest-ranked safe alternative)
```

Every line of this reasoning is returned to the interface as text and rendered in the Action Hub, so the plan arrives with its own justification attached.

09

Executive Dashboard

The landing view — a live operational snapshot answering “what needs attention right now?” It is read-only apart from one action: every alert row can launch the agent pipeline for that SKU.

FOUR KPI TILES

- **Critical low stock SKUs** — currently 53, counting products below reorder point
- **Active purchase orders** — approved and in flight
- **Average lead time** — order to shipped, across the fleet
- **Supplier reliability average** — across the unified pool

BELOW THE TILES

- **Top SKUs by deficit** — horizontal bar chart of the eight largest shortfalls
- **Recent purchase orders** — a running list of what has been approved
- **Live alert table** — every SKU below reorder point, sorted by deficit, each row with a *Trigger AI Procurement* button

The tiles animate their values on load, and each carries a tone colour keyed to severity rather than decoration — urgent counts read red, healthy metrics read green.

10

Pipeline & XAI Inspector

The explainability view, and the most distinctive part of the system. It runs the pipeline for a chosen SKU and then **replays the Knowledge Agent's reasoning as a step-by-step animation** rather than dumping a log.

PIPELINE STRIP

Four agent cards light up in sequence — idle, active, completed — each showing its summary line and confirming the stage actually ran.

FORWARD-CHAINING PLAYER

Play, Pause, Next Step, Replay, and 0.5× / 1× / 2× speed. A progress bar tracks “Step 4 of 21”, and the active card scrolls itself into view.

WHAT THE ANIMATION REVEALS

- Each **fact** as a card showing the supplier's real attribute values
- Each **rule** with its IF/THEN text, then its conditions evaluated *individually* — an AND rule shows both sides separately rather than a single verdict
- The **arithmetic comparison** made explicit: `0.0502 > 0.05 → TRUE`
- The **combination step**: `FALSE AND FALSE → RULE FAILED`
- The resulting **decision**, then a closing summary with rules evaluated, rules matched, and the reason in plain language

Rules are clickable to jump ahead, and any supplier the agent evaluated but that isn't part of the guided walkthrough is listed below as an expandable card carrying the same full detail. Every value shown is read from the agent's trace — nothing in the animation is recomputed in the browser, so the visualisation cannot drift from the actual reasoning.

11

Supplier Matrix

A standalone view of the heuristic search itself, independent of any particular SKU. It exists to make the scoring inspectable.

- **Heuristic score chart** — all ten suppliers ranked by Final Score
- **Score composition chart** — grouped bars showing the four weighted components per supplier, with each weight in the legend
- **Full matrix table** — cost, rating, delivery, reliability, defect rate, stock, and score for every supplier
- **Deficit filter** — enter a required quantity to re-run eligibility and watch the pool, the Min/Max boundaries, and the ranking all recompute

Clicking any row expands a breakdown showing precisely how that supplier's Raw Score was assembled — component score, weight, and resulting contribution:

```
SUP_5 — expanded score composition

Rating        0.832 × 0.30  =  0.250
Cost          0.974 × 0.25  =  0.243
Reliability   0.744 × 0.25  =  0.186
Delivery      1.000 × 0.20  =  0.200
                               ──────

Raw Score                      0.8791
Final Score                    100.0      Rank #1
```

The deficit filter is the clearest demonstration that normalisation respects eligibility. Set it to `7000` and `SUP_9` — the cheapest supplier in the whole pool — drops out; the cost baseline immediately shifts from ₹10.8769 to ₹10.9643, and every remaining supplier's cost score is recalculated against the narrower field.

12

Action Hub

Where a recommendation becomes a committed order. It reads the plan produced by the Planning Agent and presents it for human approval — the system recommends, a person decides.

THE PLAN UNDER REVIEW

- Recommended order quantity
- Projected delivery date
- Backup vendor
- Total cost
- Supplier status badge

SUPPORTING EVIDENCE

- **Replenishment curve** — projected stock over time, stepping up on arrival
- **Planning rationale** — the Planning Agent's own reasoning lines
- **Order history** — every previously approved purchase order

A single *Approve & Execute* action writes the purchase order to the database with its quantity, unit cost, total, arrival estimate and backup supplier, then returns the new PO number and refreshes the history table. That write is the only place the system mutates procurement state — everything before it is analysis.

13

API reference

A FastAPI service exposes the agent engine over REST; the frontend is a dependency-free single-page app that consumes it.

METHOD	ENDPOINT	PURPOSE
GET	**/api/system-health**	Uptime, per-table row counts, agent status, active alert count
GET	**/api/dashboard**	KPI tiles, alert table, recent purchase orders
GET	**/api/products**	Product search, with urgent-only filter
POST	**/api/run-agent-pipeline**	Runs all four agents for one SKU; returns every intermediate result
GET	**/api/suppliers**	Heuristic ranking; optional `required_deficit` applies eligibility filtering
POST	**/api/approve-order**	Writes an approved purchase order
GET	**/api/orders**	Purchase order history

Each ranked candidate carries a nested `scoring` object — weights, component scores, contributions, Raw Score and Final Score — which is what lets the interface explain a ranking without ever recalculating it.

14

Running the system

```
python run_app.py                  # → http://127.0.0.1:8000

python run_app.py --port 8080      # alternative port
python run_app.py --rebuild-db     # re-ingest all six CSVs from scrat

python -m pytest backend/tests/    # 34 tests covering the heuristic
```

The launcher installs missing dependencies, builds the SQLite database on first run, and starts the server. Stack: **FastAPI** and **pandas** on the backend, **SQLite** for storage, and vanilla JavaScript with **Chart.js** on the frontend — no build step, no bundler.

PROJECT LAYOUT

```
backend/
  agents/
    coordinator_agent.py    pipeline orchestration & timing
    perception_agent.py     inventory observation, deficit
    supplier_agent.py       heuristic search & ranking
    knowledge_agent.py      forward chaining & rules
    planning_agent.py       order plan & scheduling
  tests/                    heuristic test suite
  config.py                 weights & rule thresholds
  database.py               CSV ingestion & supplier pool
  main.py                   FastAPI routes
frontend/
  index.html
  static/js/app.js          all four views
  static/css/styles.css     design tokens, light & dark
```

Every tunable lives in `config.py`: the four heuristic weights, all four rule thresholds, the target-stock multiplier, and the candidate shortlist size. Changing a rule threshold there changes the reasoning everywhere — including the animated explanation — because the interface reads its values from the agent rather than holding its own copy.

InsightIQ — AI-Based Intelligent Procurement Decision Support System. All figures in this document were captured from a live run of the system; the worked example follows SKU-0717 “Headphones” end to end.